Deep Bluffing: Heads-Up Limit Texas Hold'em Poker Bot

Abhishek Marre

Roll No : 250640

18th July ,2026

1 Introduction

This project presents an autonomous poker-playing agent for Heads-Up Limit Texas Hold'em (HULHE). The objective is to design an intelligent bot capable of making decisions under imperfect information without using external poker libraries. The bot estimates winning probability using Monte Carlo simulations and selects actions using Expected Value (EV) heuristics.

2 Problem statement

The poker agent receives:

• Hole cards

• Community cards

• Pot size

• Stack size

• Amount to call

• Legal actions :

The agent must return one legal action:

• FOLD

• CALL

• RAISE

3 Methodology

The implementation consists of five major modules:

1. Card Representation

2. Deck Generation

3. Hand Evaluation

4. Monte Carlo Equity Estimation

5. Expected Value Decision Engine

4 Hand Evaluation

The evaluator determines the strongest poker hand among all possible five-card combinations. Supported hand rankings include:

• High Card

• One Pair

• Two Pair

• Three of a Kind

• Straight

• Flush

• Full House

• Four of a Kind

• Straight Flush

5 Monte Carlo Simulation

Since opponent cards are hidden, the remaining deck is sampled randomly.

For every simulation:

1. Random opponent cards are generated.

2. Remaining community cards are generated.

3. Both hands are evaluated.

4. Win percentage is estimated.

The estimated winning probability is

$$Equity = \frac{(Wins + 0.5 * Ties)}{Total\ Simulation}$$

6 Expected Value Strategy

Decision making follows the Expected Value equation:

$$EV = (Pwin \times P\ ot) - (Plose \times Cost)$$

Decision policy:

- High equity → Raise
- Medium equity → Call
- Low equity → Fold

7 Implementation

The project is implemented entirely in Python.

Modules include:

- agent.py
- card.py
- deck.py
- hand evaluator.py
- equity calculator.py

No external poker libraries are used.

8 Conclusion

The developed poker bot successfully evaluates poker hands, estimates winning probability using Monte Carlo simulation, and selects legal actions using Expected Value heuristics. The implementation follows all assignment constraints while remaining modular, efficient, and suitable for tournament evaluation.